

</> Bash



```
mkdir proyecto-spa  
cd proyecto-spa  
npm init -y
```

PASO 2. Instalar dependencias

Json Server

</> Bash



```
npm install json-server@0.17.4 --save-dev
```

Verificar versión:

</> Bash



```
npx json-server --version
```

Debe mostrar:

</> Bash



```
0.17.4
```

Serve

Para levantar el frontend

❏ Bash



```
npm install serve --save-dev
```

Concurrently

Para ejecutar API y Frontend al mismo tiempo

❏ Bash



```
npm install concurrently --save-dev
```

PASO 3. Crear estructura

proyecto-spa/



├─ db.json

├─ package.json

├─ index.html

└─ src/

├─ main.js

├─ services/

|

├─ views/

|

├─ components/

|

└─ utils/

PASO 4. Crear db.json

Usuarios

JSON

```
{
  "users": [
    {
      "id": 1,
      "name": "Admin",
      "email": "admin@test.com",
      "password": "123456",
      "role": "admin"
    }
  ],
  "projects": []
}
```

PASO 5. Configurar package.json

JSON



```
{  
  "scripts": {  
    "api": "json-server --watch db.json --port 3000",  
  
    "dev": "serve .",  
  
    "start": "concurrently \"npm run api\" \"npm run dev\""  
  }  
}
```

PASO 6. Ejecutar proyecto

Solo API

 Bash

```
npm run api
```



Solo Front

 Bash

```
npm run dev
```



Todo junto

 Bash

```
npm start
```



PASO 7. URL BASE

En api.js

 JavaScript

```
const URL_API =  
  "http://localhost:3000";
```



PASO 6. Ejecutar proyecto

Solo API

```
</> Bash  
npm run api
```



Solo Front

```
</> Bash  
npm run dev
```



Todo junto

```
</> Bash  
npm start
```



PASO 7. URL BASE

En api.js

```
</> JavaScript  
const URL_API =  
  "http://localhost:3000";
```



PASO 8. Login

Capturar formulario

JavaScript



```
const email =  
  document.querySelector("#email");  
  
const password =  
  document.querySelector("#password");
```

Buscar usuario

JavaScript



```
const respuesta =  
  await fetch(  
    `${URL_API}/users?email=${email}`  
  );  
  
const usuarios =  
  await respuesta.json();
```

Validar

JavaScript



```
if(
  usuarios.length > 0 &&
  usuarios[0].password === password
){
  console.log("Login correcto");
}
```

PASO 9. Guardar sesión

JavaScript



```
localStorage.setItem(
  "usuario",
  JSON.stringify(usuario)
);
```

PASO 10. Recuperar sesión

JavaScript



```
const usuario =
  JSON.parse(
    localStorage.getItem(
      "usuario"
    )
  );
```



PASO 11. Cerrar sesión

«» JavaScript



```
localStorage.removeItem(  
  "usuario"  
);
```

PASO 12. Router SPA

Escuchar cambios de ruta

«» JavaScript



```
window.addEventListener(  
  "hashchange",  
  router  
);  
  
window.addEventListener(  
  "load",  
  router  
);
```

Obtener ruta actual

«» JavaScript



```
const ruta =  
  location.hash;
```



PASO 13. GET

Obtener todos

JavaScript



```
async function obtenerProyectos() {  
  
  const respuesta =  
    await fetch(  
      `${URL_API}/projects`  
    );  
  
  const datos =  
    await respuesta.json();  
  
  return datos;  
}
```

PASO 14. GET POR ID

JavaScript



```
async function obtenerProyecto(id){  
  
  const respuesta =  
    await fetch(  
      `${URL_API}/projects/${id}`  
    );  
  
  return await respuesta.json();  
}
```



PASO 15. POST

Crear

«» JavaScript



```
async function crearProyecto(proyecto){

  await fetch(
    `${URL_API}/projects`,
    {
      method: "POST",

      headers: {
        "Content-Type":
          "application/json"
      },

      body:
        JSON.stringify(proyecto)
    }
  );
}
```

PASO 16. PUT

Actualizar todo

JavaScript



```
async function actualizarProyecto(
  id,
  proyecto
){

  await fetch(
    `${URL_API}/projects/${id}`,
    {
      method: "PUT",

      headers: {
        "Content-Type":
          "application/json"
      },

      body:
        JSON.stringify(proyecto)
    }
  );
}
```

PASO 17. PATCH

Actualizar parcialmente

JavaScript



```
async function cambiarEstado(  
  id,  
  estado  
) {  
  
  await fetch(  
    `${URL_API}/projects/${id}`,  
    {  
      method: "PATCH",  
  
      headers: {  
        "Content-Type":  
          "application/json"  
      },  
  
      body:  
        JSON.stringify({  
          status: estado  
        })  
    }  
  );  
}
```

PASO 18. DELETE

JavaScript



```
async function eliminarProyecto(id){

    await fetch(
        `${URL_API}/projects/${id}`,
        {
            method: "DELETE"
        }
    );
}
```

PASO 19. Pintar datos

JavaScript



```
const contenedor =
    document.querySelector(
        "#contenedor"
    );

contenedor.innerHTML =
    proyectos.map(
        proyecto => `
        <div>

            <h3>
                ${proyecto.name}
            </h3>

        </div>
        `
    ).join("");
```



PASO 20. Flujo completo de examen

Si te bloqueas mañana, recuerda esto:

1. Crear carpetas
2. Crear db.json
3. Instalar dependencias
4. Configurar package.json
5. Levantar json-server
6. Crear api.js
7. Crear login
8. Crear router
9. Crear vista principal
10. GET
11. POST
12. PUT/PATCH
13. DELETE
14. LocalStorage
15. Logout
16. Probar todo



⚠ Errores que más salen en la prueba

Error 404

Ruta mal escrita

📄

Error CORS

URL incorrecta

📄

Error al hacer POST

Falta:

⌞ JavaScript

headers:{
 "Content-Type":
 "application/json"
}

📄

Error al guardar

Olvidaste:

⌞ JavaScript

JSON.stringify()

📄

Error al recuperar

Olvidaste:

JavaScript



JSON.parse()

